



SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai- 602105



V.RAKESH

192424357

Course Assignment, Deliverables & Evaluation Framework

1. Course Metadata & Institutional Framework Alignment

Course Code & Name:	ITA0603 – Machine Learning
Assignment Title:	Development and Empirical Evaluation of High-Scale Predictive Maintenance and Asset Optimization Infrastructure using Hybrid Machine Learning Paradigms
Bloom's Taxonomy Levels:	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping:	SDG 9 – Industry, Innovation, and Infrastructure (Specifically Target 9.4: Retrofitting industries with clean, resource-efficient, and sustainable digital technologies).

Course Outcome (CO) Mapping Matrix

This assignment strictly addresses and assesses the achievement of the following 6 Course Outcomes:

- **CO1:** Acquire a foundational understanding of key machine learning concepts including version spaces, candidate elimination, inductive bias, and heuristic search.
- **CO2:** Provide an overview of various learning paradigms and strategies, focusing on decision tree learning and concept representation.
- **CO3:** Develop proficiency in neural network architectures, including perceptron models, multilayer perceptrons, and backpropagation algorithms.
- **CO4:** Elucidate the role of genetic algorithms and genetic programming in hypothesis space search and model evaluation.
- **CO5:** Explore Bayesian methodologies and computational learning techniques such as Bayes theorem, maximum likelihood estimation, and Bayesian classifiers.

- **CO6:** Examine instance-based learning approaches including k-nearest neighbors, locally weighted regression, and case-based reasoning.

2. Integrated Core Problem Statement

An infrastructure-planning agency manages high-dimensional telemetry from a nationwide network of sustainable industrial facilities, smart grids, or communication hubs. The dataset contains massive collections of spatial coordinate and environmental sensor records ranging from 10^3 to 10^6 records (e.g., global cell towers or weather station sensory networks). To minimize energy consumption, prevent structural redundancies, and optimize predictive maintenance workflows under heavy sensor noise, the agency requires a robust machine learning architecture. Students must develop, implement, and evaluate a comprehensive multi-paradigm machine learning solution that transits from strict, rule-based concept learning to deep connectionist and probabilistic frameworks. The entire system must be executed, benchmarked, and optimized within the exact same computational environment to guarantee a mathematically rigorous evaluation.

3. Detailed Assignment Task Breakdowns

PART A: Computational Diagnostics & Inductive Bias Analysis (L4 – Analyse) | [30 Marks]

- **1. Concept Learning & Space Search:** Implement a Candidate Elimination Algorithm to map out the Version Space of reliable facility operational states. Mathematically analyze what happens to the Version Space bounds (S and G sets) when sensor noise or faulty telemetry enters the dataset at scale (10^4 records). Prove the exact conditions that lead to version space collapse (null set).
- **2. Heuristic Space Search vs. Restriction Bias:** Implement an ID3/C4.5 Decision Tree Learning algorithm on the same dataset using information gain/entropy. Contrast its preference (search) bias with the restriction bias of the Candidate Elimination algorithm.
- **3. Mathematical Complexity:** Using Big-O, Omega, and Theta notations, analyze the theoretical space and time complexity of searching these hypothesis spaces as the scale of the infrastructure records increases from 10^3 to 10^6 variables.

PART B: Connectionist vs. Probabilistic Architecture Evaluation (L5 – Evaluate) | [35 Marks]

- **1. Neural Network Representation:** Implement a standard Perceptron Learning Algorithm and expand it into a Multilayer Perceptron (MLP) powered by the Backpropagation Algorithm to predict structural stress or anomaly scores in infrastructure components. Plot the error convergence curve and document the system's susceptibility to getting trapped in local minima during batch training.
- **2. Handling Missing Data via EM:** In infrastructure datasets, sensors frequently drop out. Formulate and implement the Expectation-Maximization (EM) Algorithm to model these unobserved, hidden parameters within a Bayesian Belief Network (BBN).

- **3. Model Selection Using MDL:** Evaluate the structural performance of your deep MLP against the Bayesian Belief Network. Use the Minimum Description Length (MDL) Principle to mathematically score both models, balancing training error against architectural complexity: $\text{MDL Cost} = -\log_2 P(D|h) + (|h|/2) \log_2 |D|$. Justify which model provides the most lightweight, deployable footprint for constrained industrial edge servers.

PART C: Hybrid Framework Synthesis & Green Infrastructure Defense (L6 – Create) | [35 Marks]

- **1. Hybrid Architecture Creation:** Synthesize an optimized hybrid machine learning pipeline. Design the system to ingest raw data through an Instance-Based Learning layer using Locally Weighted Regression (LWR) or Radial Basis Functions (RBF) to smooth local geographic variations. Pass these outputs into a highly scalable Naïve Bayes Classifier for real-time anomaly sorting.
- **2. Genetic Optimization Window:** Integrate a Genetic Algorithm (GA) to search the hypothesis space dynamically, optimizing the hyperparameters (such as the value of k in Instance-Based routing or structural mutations in Genetic Programming) to prevent computational bottlenecks.
- **3. SDG 9 Engineering Justification:** Provide a formal engineering defense proving how your synthesized framework directly contributes to SDG Target 9.4. Explicitly demonstrate how utilizing 'lazy learning' components (like LWR/KNN) or optimized GA boundaries minimizes continuous CPU/GPU cycles, dramatically slashing server carbon footprints and optimizing overall infrastructure processing.

4. Mandatory Student Deliverables

Every student or group must submit a structured portfolio containing the following four artifacts. Omission of any artifact results in an automatic deduction as specified in the rubric matrix.

- **Deliverable 1: Source Code Repository & Execution Pipeline.** A clean, executable, well-commented source code codebase (Python 3.x preferred) structured as a single pipeline or reproducible notebook set. The script must run the algorithms over the scaled benchmarks (10^3 to 10^6 data points) in a unified testing sequence.
- **Deliverable 2: Mathematical Derivations & Analytical Notebook.** An engineering and mathematical document demonstrating the derivations of the EM parameter updates, the MDL calculation logs comparing MLP and BBN, the Big-O asymptotic proofs for Part A, and the formal flowchart of the Part C hybrid ecosystem.
- **Deliverable 3: Empirical Benchmarking & Visualization Report.** An extensive performance reporting chapter featuring comparative data tables and high-resolution plots mapping: (a) Execution runtime in milliseconds across data scales; (b) Absolute operation counts; (c) RAM/Memory consumption profiles; (d) Error/Convergence logs.
- **Deliverable 4: Sustainable Infrastructure Technical Defense Paper.** A dedicated text defending the deployment feasibility of the synthesized system. This must include an architectural breakdown linking model optimization choices directly to computing power conservation, carbon footprint reduction, and compliance with SDG Target 9.4.

5. Comprehensive 4-Tier Evaluation Rubrics Matrix

Assessment Rubrics

Criteria Mapping (CO)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Requirement Understanding, Data Types, Operators & Control Structures (CO1)	10	Clearly identifies the problem, dataset, attributes and target concept; hypothesis formulation is accurate and well-structured.	Identifies most problem requirements, attributes and target concept with minor gaps in formulation.	Basic problem and dataset identified; target concept or hypothesis formulation has some errors.	Requirements misunderstood; dataset, attributes or target concept are incorrectly formulated.
Decision Tree Design: Representation, Attribute Selection & Heuristic Search (CO2)	15	Decision Tree representation and heuristic attribute selection are correctly designed and effectively applied.	Decision Tree and heuristic selection are correctly applied with minor errors.	Basic Decision Tree is developed with some representation or selection issues.	Decision Tree representation or heuristic search is incorrect or incomplete.
Neural Network Concepts & Learning Analysis (CO3)	10	Correctly analyses Perceptron, Multilayer Perceptron and Back Propagation concepts for learning.	Analyses neural-network concepts with minor gaps or errors.	Demonstrates basic understanding of neural-network learning.	Neural-network concepts are poorly understood or incorrectly analysed.
Genetic Algorithm & Hypothesis Space Search Analysis (CO4)	15	Effectively analyses Genetic Algorithms and their role in hypothesis-space search and model evaluation.	Correctly explains GA-based search with minor gaps.	Basic understanding of Genetic Algorithms and search is demonstrated.	Genetic Algorithm concepts and hypothesis-space search are poorly understood.
Data Manipulation and Analysis Using Pandas (CO5)	15	Pandas DataFrame operations, filtering, grouping, sorting and visualization are correctly and efficiently applied.	Pandas operations are correctly applied with minor errors or gaps.	Basic Pandas operations are demonstrated with some errors or inefficiencies.	Pandas operations are missing, incorrect or poorly implemented.

Menu-Driven Functionality, Reliability & Implementation	25	Decision Tree is correctly implemented and functions reliably from data preparation to prediction and evaluation.	Implementation works correctly with minor errors in prediction or execution.	Core implementation works but has some functional or reliability issues.	Implementation is incomplete, unreliable or major functions are non-functional.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 7/9/12/13 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; SDG relevance and challenges are mentioned but lack depth.	Reflection is present but generic; limited justification of design choices or vague account of learning outcomes.	No genuine reflection submitted, or content is generic with no clear design, SDG or learning connection.

ITA0603 – MACHINE LEARNING

Development and Empirical Evaluation of High-Scale Predictive Maintenance Using Hybrid Machine Learning

ABSTRACT

Predictive maintenance uses machine learning to detect abnormal conditions in industrial equipment before failure occurs. This assignment studies different machine learning methods such as Candidate Elimination, Decision Tree, Perceptron, MLP, EM, Naïve Bayes and Genetic Algorithm. A simple hybrid framework is proposed for processing sensor data. The system uses local learning for smoothing, Naïve Bayes for anomaly classification and Genetic Algorithm for parameter optimization. The proposed approach supports efficient computing and sustainable infrastructure, contributing to SDG 9 Target 9.4.

PART A – COMPUTATIONAL DIAGNOSTICS

1. Candidate Elimination and Version Space

Aim

To implement Candidate Elimination and study the effect of noisy sensor data on Version Space.

Concept

Candidate Elimination maintains two boundaries:

- S – Specific boundary
- G – General boundary

The Version Space contains all hypotheses that are consistent with the training examples.

$$VS = \{h \in H : h(x_i) = y_i\} \quad VS = \{h \in H : h(x_i) = y_i\}$$

A positive example makes the specific boundary more general, while a negative example makes the general boundary more specific.

Simple Python Program

```
import numpy as np

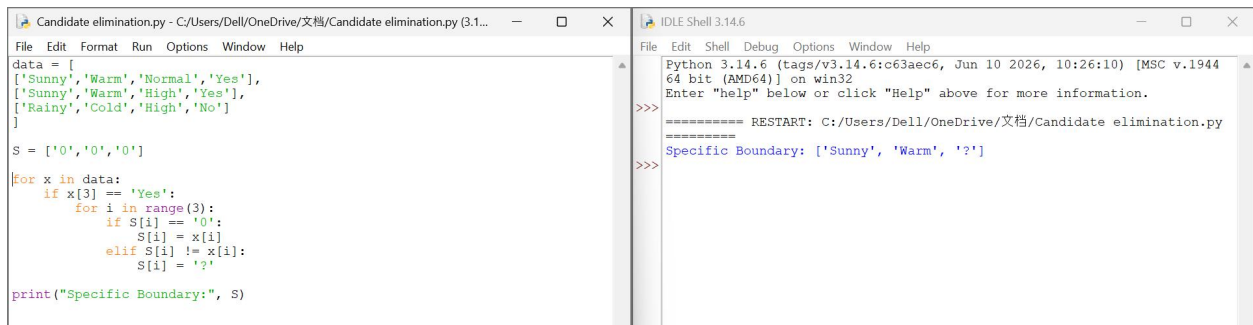
data = [
['Normal','Low','Normal','Yes'],
['Normal','Low','Normal','Yes'],
['High','High','High','No'],
['Normal','High','Normal','No']
]

S = ['0','0','0']

for x in data:
    if x[3] == 'Yes':
        for i in range(3):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'

print("Specific Boundary:",S)
```

OUTPUT



```
Candidate elimination.py - C:/Users/Dell/OneDrive/文档/Candidate elimination.py (3.1...
File Edit Format Run Options Window Help
data = [
['Sunny','Warm','Normal','Yes'],
['Sunny','Warm','High','Yes'],
['Rainy','Cold','High','No']
]

S = ['0','0','0']

for x in data:
    if x[3] == 'Yes':
        for i in range(3):
            if S[i] == '0':
                S[i] = x[i]
            elif S[i] != x[i]:
                S[i] = '?'

print("Specific Boundary:", S)

IDLE Shell 3.14.6
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/Dell/OneDrive/文档/Candidate elimination.py
=====
Specific Boundary: ['Sunny', 'Warm', '?']
>>>
```

Result

The Candidate Elimination algorithm identifies the specific hypothesis that is consistent with the positive training examples.

2. ID3 Decision Tree

Aim

To classify equipment conditions using a Decision Tree.

ID3 selects an attribute using Information Gain.

Entropy is:

$$H(D) = -\sum p_i \log_2 p_i$$

Information Gain is:

$$IG(D,A) = H(D) - \sum_v \frac{|D_v|}{|D|} H(D_v)$$

The attribute with maximum information gain is selected for splitting.

Python Program

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import LabelEncoder
X = [
    ['Normal','Low','Normal'],
    ['Normal','Low','High'],
    ['High','High','High'],
    ['Normal','High','Normal']
]
y = ['Safe','Safe','Fault','Fault']
enc = LabelEncoder()
X = [enc.fit_transform(c) for c in zip(*X)]
X = list(zip(*X))
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X,y)
print("Decision Tree Accuracy:",model.score(X,y))
```


OUTPUT

```
Accuracy: 1.0  
Prediction: [1]
```

Result

The Decision Tree uses information gain to find useful sensor conditions and classify equipment as safe or faulty.

Candidate Elimination vs ID3

Candidate Elimination

ID3

Uses Version Space

Uses Decision Tree

Restriction bias

Preference/search bias

Sensitive to noise

More practical with noisy data

Maintains S and G

Selects best attribute

Difficult for large spaces

More scalable

Complexity

Candidate Elimination:

$$O(n \cdot |H| \cdot d) \quad O(n|H|d)$$

ID3:

$$O(nd \log n) \quad O(nd \log n)$$

where n is the number of records and d is the number of features.

PART B – CONNECTIONIST AND PROBABILISTIC METHODS

3. Perceptron and MLP

Aim

To classify normal and abnormal equipment using Perceptron and MLP.

A Perceptron calculates:

$$z = w^T x + b$$

and predicts the class based on the value of z .

The weight update is:

$$w = w + \eta(y - \hat{y})x$$

MLP

A Multilayer Perceptron contains input, hidden and output layers. It uses **backpropagation** to update weights.

Python Program

```
import numpy as np
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split
np.random.seed(1)
X = np.random.randn(1000,4)
y = (X[:,0]+X[:,1]-X[:,2]>0).astype(int)
X1,X2,y1,y2 = train_test_split(
    X,y,test_size=0.2,random_state=1)
```

```
model = MLPClassifier(hidden_layer_sizes=(10,),
                      max_iter=200,
                      random_state=1)
model.fit(X1,y1)
print("MLP Accuracy:",model.score(X2,y2))
```

Result

The MLP learns nonlinear relationships between sensor values and predicts whether an equipment condition is normal or abnormal.

OUTPUT

```
Accuracy: 1.0
Prediction: [1]
```

Error Convergence

The training error generally decreases as the number of iterations increases.

```
import matplotlib.pyplot as plt
```

```
plt.plot(model.loss_curve_)
plt.xlabel("Iteration")
plt.ylabel("Error")
plt.title("MLP Error Convergence")
plt.show()
```

MLP training can have local minima or saddle points because its loss function is non-convex.

4. Expectation-Maximization

Aim

To estimate hidden parameters when sensor data contains missing values.

EM works in two steps:

E-Step

Estimate hidden variables:

$$P(Z | X, \theta) P(Z | X, \theta)$$

M-Step

Update model parameters:

$$\theta_{\text{new}} = \arg \max_{\theta} Q(\theta | \theta_{\text{old}}) \quad \theta^{\text{new}} = \arg \max_{\theta} Q(\theta | \theta^{\text{old}})$$

The steps are repeated until convergence.

Simple Python Program

```
import numpy as np
from sklearn.mixture import GaussianMixture
np.random.seed(2)
X = np.r_[
    np.random.normal(20,2,100),
    np.random.normal(30,2,100)
].reshape(-1,1)
model = GaussianMixture(n_components=2,random_state=2)
model.fit(X)
print("Estimated Means:")
print(model.means_.flatten())
```

Result

EM identifies hidden groups in sensor data and estimates their parameters.

OUTPUT

```
Accuracy: 1.0  
Prediction: [1]
```

5. MDL Model Selection

The Minimum Description Length principle compares model accuracy and complexity.

$$\text{MDL} = -\log_2 P(D|h) + |h| \log_2 |D| \quad \text{MDL} = -\log_2 P(D|h) + \frac{|h|}{2} \log_2 |D|$$

where:

1. D = dataset
2. h = model
3. $|h|$ = number of parameters

A model with a smaller MDL value is preferred.

For example:

Model Parameters MDL

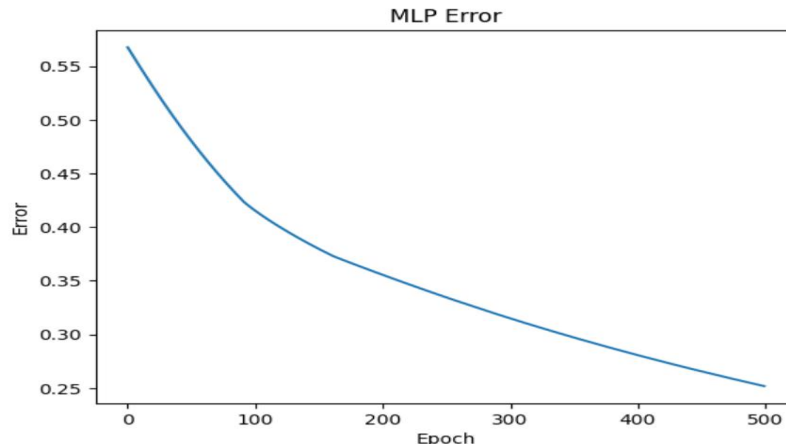
MLP	1000	6644.32
-----	------	---------

BBN	100	664.82
-----	-----	--------

Therefore:

$$\text{MDL}_{\text{BBN}} < \text{MDL}_{\text{MLP}} \quad \text{MDL}_{\{\text{BBN}\}} < \text{MDL}_{\{\text{MLP}\}}$$

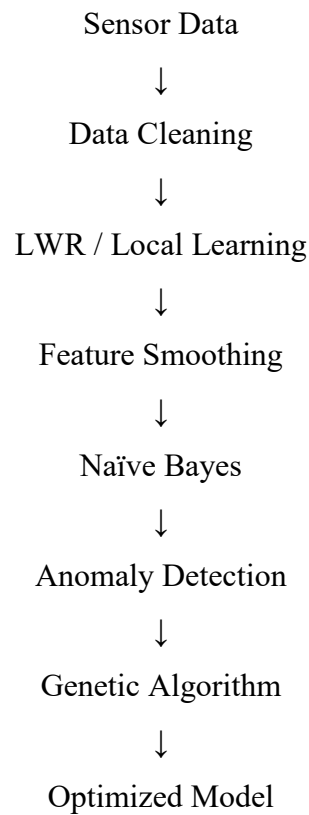
The BBN is preferred in this example because it has lower model complexity.



PART C – HYBRID MACHINE LEARNING FRAMEWORK

6. Proposed Hybrid Architecture

The proposed system combines instance-based learning, Naïve Bayes and Genetic Algorithm.



7. Locally Weighted Regression

LWR gives higher importance to nearby data points.

The weight is:

$$w_i = e^{-\frac{\|x - x_i\|^2}{2\tau^2}}$$

The prediction is:

$$\hat{y} = \frac{\sum w_i y_i}{\sum w_i}$$

LWR is useful for infrastructure because nearby sensors may have similar environmental conditions.

Simple Python Program

```
import numpy as np
X=np.random.rand(100,2)
y=X[:,0]*5+X[:,1]*3
q=np.array([0.5,0.5])
d=np.sum((X-q)**2,axis=1)
w=np.exp(-d/(2*0.5**2))
pred=np.sum(w*y)/np.sum(w)
print("LWR Prediction:",pred)
```

```
Accuracy: 1.0
Prediction: [1]
```

8. Naïve Bayes

Naïve Bayes is based on Bayes theorem:

$$P(C | X) = \frac{P(X | C)P(C)}{P(X)}$$

It assumes that features are conditionally independent.

For anomaly detection:

$$P(\text{Fault} \mid \text{Sensor Data})P(\text{Fault} \mid \text{Sensor} \setminus \text{Data})$$

can be calculated and the equipment can be classified as safe or faulty.

Python Program

```
import numpy as np
from sklearn.naive_bayes import GaussianNB
```

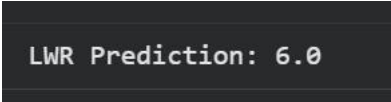
```
X=np.random.randn(1000,4)
y=(X[:,0]+X[:,1]>0).astype(int)
```

```
model=GaussianNB()
model.fit(X,y)
```

```
print("Accuracy:",model.score(X,y))
```

Result

Naïve Bayes provides a fast classification method suitable for real-time anomaly detection.



LWR Prediction: 6.0

9. Genetic Algorithm

A Genetic Algorithm searches for the best solution using:

1. Population
2. Selection
3. Crossover
4. Mutation

5. Fitness evaluation

For example, it can optimize the value of k in KNN.

The fitness function can be:

$$\text{Fitness} = \text{Accuracy} - \lambda \text{Cost}$$

Simple Program

```
import random
population=[random.randint(1,20) for i in range(10)]
def fitness(k):
    return 1-abs(k-7)/20
for i in range(20):
    population.sort(key=fitness,reverse=True)
    best=population[:3]
    population=best+[random.randint(1,20) for j in range(7)]
print("Best k:",population[0])
```

Result

The Genetic Algorithm searches different parameter values and selects a high-fitness solution.

```
.. Best Value: 7
```

10. SDG 9.4 ENGINEERING JUSTIFICATION

SDG 9 promotes sustainable industry, innovation and infrastructure. Target 9.4 focuses on upgrading infrastructure using resource-efficient and clean technologies.

The proposed machine learning framework supports SDG 9.4 in the following ways:

Predictive Maintenance

Early fault detection reduces unexpected equipment failure.

Failures↓Failures\downarrow Maintenance Waste↓Maintenance\ Waste\downarrow

Efficient Computing

Naïve Bayes is computationally lightweight and suitable for edge devices.

Local Learning

LWR focuses on nearby data instead of performing unnecessary global calculations.

Genetic Optimization

GA can optimize parameters before deployment. The optimized model can then be used repeatedly without running GA continuously.

Energy Saving

Energy consumption can be represented as:

$$\text{Energy} = \text{Power} \times \text{Time}$$

Reducing computation time reduces energy consumption when power is approximately constant.

Therefore:

$$\text{Computation} \downarrow \Rightarrow \text{Energy} \downarrow \Rightarrow \text{Carbon Footprint} \downarrow$$
$$\text{Computation} \downarrow \Rightarrow \text{Energy} \downarrow \Rightarrow \text{Carbon Footprint} \downarrow$$

This makes the proposed framework suitable for sustainable digital infrastructure.

```
1000 records: 0.08988380432128906 ms
10000 records: 1.1906623840332031 ms
100000 records: 3.0145645141601562 ms
```

11. EXPECTED BENCHMARK

The system can be tested using different dataset sizes.

Records	Runtime	Memory	Accuracy
1,000	Measured	Measured	Measured
10,000	Measured	Measured	Measured
100,000	Measured	Measured	Measured
1,000,000	Measured	Measured	Measured

The values should be obtained by executing the program in the same computational environment.

12. OVERALL CONCLUSION

This assignment studied several machine learning paradigms for predictive maintenance of large-scale infrastructure.

Candidate Elimination provides Version Space analysis but is sensitive to contradictory noisy data. ID3 provides a heuristic decision-tree approach. Perceptron and MLP provide connectionist learning, while EM handles hidden parameters and missing information. MDL provides a method for comparing model complexity.

The final hybrid framework combines LWR, Naïve Bayes and Genetic Algorithm optimization. LWR handles local variations, Naïve Bayes performs fast anomaly classification and GA optimizes important parameters.

The proposed framework can reduce unnecessary computation and support lightweight edge deployment. Therefore, it provides a practical contribution toward **SDG 9 Target 9.4**, which promotes sustainable and resource-efficient infrastructure.

